

## AI ADAPTIVE STUDY MATERIAL RECOMMENDER SYSTEM

---

**Project Report** Submitted in Partial Fulfilment of the Requirements for the Course Project

---

| Field                  | Details                                                                                                     |
|------------------------|-------------------------------------------------------------------------------------------------------------|
| <b>Project Title</b>   | AI Adaptive Study Material Recommender System                                                               |
| <b>Author</b>          | Sayan Mondal, 25BAI11532                                                                                    |
| <b>Programme</b>       | B.Tech — Computer Science & Engineering (AIML)                                                              |
| <b>Year / Semester</b> | 1st Year                                                                                                    |
| <b>Institution</b>     | VIT Bhopal University                                                                                       |
| <b>Repository</b>      | <a href="https://github.com/Sayan-stg/Study_Recommender">https://github.com/Sayan-stg/Study_Recommender</a> |
| <b>Date</b>            | March 2026                                                                                                  |

---

### Abstract

The exponential growth of digital educational content has created a paradox of choice for learners: an overwhelming volume of study material with no intelligent mechanism to navigate it. This project presents the **AI Adaptive Study Material Recommender System**, a Python-based intelligent agent that addresses this challenge by recommending curated study materials — videos, notes, and practice problems — tailored to each student's subject, preferred difficulty level, and material type.

The system is architected around a four-behaviour intelligent agent model: **Percept**, **Brain**, **Action**, and **Learn**. It employs a **constraint-based search strategy** for efficient dataset filtering and a **heuristic ranking formula** to score and order results by predicted utility. Crucially, the system implements an **adaptive learning mechanism** inspired by reinforcement feedback, leveraging user ratings (on a scale of 1–5) to continuously recalibrate material scores, thereby improving the relevance of future recommendations. Built entirely with Python and Pandas — without dependency on advanced machine learning libraries — this project demonstrates how core AI/ML principles can be effectively realised in a lightweight, interpretable system.

---

## 1. Introduction

### 1.1 Background and Problem Statement

Modern students are confronted with a vast and fragmented landscape of study resources. Platforms such as YouTube, NPTEL, GeeksforGeeks, and institutional portals collectively host thousands of materials on any given academic subject. Without an intelligent filtering and ranking mechanism, students must spend significant time evaluating resources manually, often defaulting to arbitrary choices rather than optimal ones. This inefficiency directly impacts learning outcomes.

A **recommender system** — a class of information filtering systems that seeks to predict a user's preference for items — offers a principled solution. While large-scale recommender systems (e.g., Netflix, Spotify) rely on collaborative filtering, deep learning embeddings, and massive datasets, the underlying concepts of search, scoring, and adaptive personalisation can be effectively demonstrated in a smaller, controlled academic context.

### 1.2 Motivation

This project was motivated by the desire to:

- **Apply core AI/ML concepts tangibly**, demonstrating that intelligent behaviour does not exclusively require black-box neural networks.
- **Design a system that genuinely improves over time**, mirroring the feedback loops fundamental to reinforcement learning.
- **Build a fully functional, end-to-end application** that a real student could use to navigate study materials.

### 1.3 Objectives

The primary objectives of this project are as follows:

1. To design and implement an **intelligent agent** that models the perception–action–learning cycle for educational recommendations.
2. To apply a **constraint-based search strategy** to efficiently filter study materials from a structured dataset.
3. To implement a **heuristic ranking formula** that assigns a meaningful priority score to each filtered material.

4. To incorporate an **adaptive learning mechanism** that adjusts material scores based on accumulated user feedback, enabling the system to personalise recommendations over time.
5. To develop a clean, well-structured **Command Line Interface (CLI)** for user interaction.

## 1.4 Scope

The project operates on a curated, locally maintained CSV dataset of study materials. It covers subjects available within the dataset, with three difficulty tiers (Easy, Medium, Hard) and three material types (Video, Notes, Practice). The system persists all user feedback across sessions, continuously refining its internal scoring model.

---

---

## 2. System Architecture & Modules

### 2.1 High-Level Architecture

The system follows a **modular, layered architecture** in which each Python module is assigned a single, well-defined responsibility. The `main.py` entry point orchestrates the flow, delegating all intelligence to the `StudyAgent` class (`agent.py`), which in turn collaborates with the `recommender.py` and `feedback.py` modules. The `utils.py` module provides shared utility functions accessible to all layers.

#### System Interaction Flow:

Student (CLI Input)

|

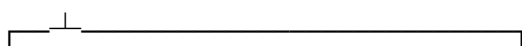


[ `main.py` ] — Orchestrates the program loop

|



[ `agent.py` ] — StudyAgent: Perceive → Recommend → Learn

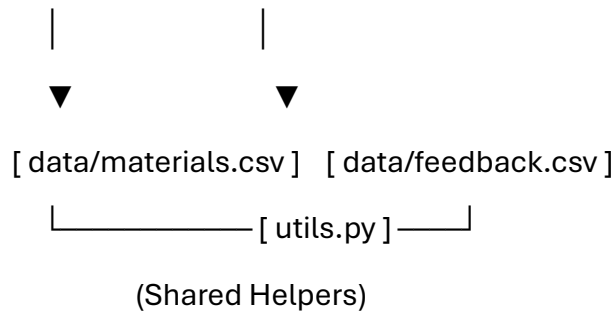


[ `recommender.py` ]

[ `feedback.py` ]

Search + Rank

Adaptive Learning



### Data Flow Summary:

1. The student provides their **preferences** (subject, difficulty, type) via the CLI.
2. The StudyAgent passes these constraints to `recommender.py`, which filters `materials.csv` and applies heuristic scoring — using adaptive scores fetched from `feedback.py`.
3. The top-ranked results are **displayed** to the student.
4. The student optionally **rates** a material (1–5 stars).
5. The rating is **appended** to `feedback.csv` by `feedback.py`, immediately influencing future scoring computations.

---

## 2.2 Module Descriptions

### 2.2.1 `main.py` — Program Orchestrator

`main.py` serves as the application's **entry point and session loop controller**. It initialises the system, renders the welcome banner, instantiates the StudyAgent, and drives the interactive main menu.

#### Key Responsibilities:

- Rendering the application banner via `print_banner()`.
- Presenting a three-option menu: *(1) Get Recommendations, (2) View Feedback & Learning Progress, (3) Exit*.
- Calling the agent's `perceive()`, `recommend()`, and `collect_feedback()` methods in sequence for option 1.
- Calling `show_learning_progress()` for option 2.
- Maintaining the session loop until the user chooses to exit.

**Design Note:** By keeping `main.py` as a thin orchestrator, all business logic remains contained within the agent and supporting modules, adhering to the **Separation of Concerns** principle.

---

### 2.2.2 agent.py — The Intelligent Agent (StudyAgent)

agent.py implements the StudyAgent class, which encapsulates the **four core behaviours of an intelligent agent**:

| Behaviour      | Method                               | Description                                                                                   |
|----------------|--------------------------------------|-----------------------------------------------------------------------------------------------|
| <b>Percept</b> | perceive()                           | Reads and validates student input (subject, difficulty, type) from the environment (CLI).     |
| <b>Brain</b>   | <i>(Delegated to recommender.py)</i> | Executes the constraint-based search and heuristic ranking.                                   |
| <b>Action</b>  | recommend()                          | Displays the ranked list of recommended materials to the student.                             |
| <b>Learn</b>   | collect_feedback()                   | Solicits a rating from the student and triggers the adaptive learning update via feedback.py. |

#### **perceive() Method:**

- Dynamically retrieves valid subjects from the dataset via `get_available_options()`.
- Implements **input validation loops** for each parameter, ensuring only valid inputs are accepted before proceeding.
- Returns a tuple (subject, difficulty, mat\_type) representing the student's constraint set.

#### **recommend() Method:**

- Invokes `search_materials(subject, difficulty, mat_type)` from `recommender.py`.
- Displays results using `display_materials_table()` from `utils.py`.
- Stores results in `self.last_recommendations` to enable the subsequent feedback step.

#### **collect\_feedback() Method:**

- Prompts the student to optionally rate one of the recommended materials.
- Retrieves the selected material's ID and metadata from `self.last_recommendations`.
- Calls `save_feedback()` in `feedback.py`, which appends the rating to persistent storage and triggers adaptive score recalculation on the next session.

---

### 2.2.3 recommender.py — Search Engine & Heuristic Ranker

recommender.py is the system's **analytical core**, responsible for implementing the two primary AI concepts: constraint-based search and heuristic ranking.

#### **search\_materials(subject, difficulty, mat\_type) Function:**

This function implements a structured four-step pipeline:

1. **Load Data:** Loads materials.csv into a Pandas DataFrame via load\_csv().
2. **Update Adaptive Scores:**  
Calls update\_scores\_in\_materials() from feedback.py to refresh each material's score based on the latest user feedback before filtering.
3. **Constraint Filtering (Search Strategy):** Filters the DataFrame using case-insensitive equality constraints on subject, difficulty, and type.
4. **Heuristic Scoring & Sorting:** Applies calculate\_heuristic\_score() to each row and sorts results in descending order of the final score.

#### **calculate\_heuristic\_score(adaptive\_score, difficulty, mat\_type) Function:**

Applies the heuristic ranking formula using the pre-defined weight dictionaries:

python

```
DIFFICULTY_WEIGHTS = { "Easy": 1.0, "Medium": 1.1, "Hard": 1.2 }
```

```
TYPE_WEIGHTS      = { "Video": 1.0, "Notes": 1.05, "Practice": 1.1 }
```

#### **get\_available\_options() Function:**

Queries the dataset to return dynamically derived lists of unique subjects, along with the fixed lists of difficulty levels and material types. This ensures that the menu always reflects the actual contents of the dataset.

---

### 2.2.4 feedback.py — Adaptive Learning Engine

feedback.py is the system's **intelligence amplifier**. It manages the persistence of user ratings and implements the adaptive score recalculation formula that enables the system to learn over time.

#### **save\_feedback(...) Function:**

Appends a new feedback record to data/feedback.csv. Each record contains:

- material\_id, title, subject, difficulty, type

- rating (integer, 1–5)
- timestamp (ISO-format datetime string)

The function safely handles both the creation of a new feedback file and appending to an existing one.

**calculate\_adaptive\_score(material\_id, base\_score) Function:**

Implements the core **adaptive learning formula** (detailed in Section 3.2). It reads all historical ratings for a specific material from feedback.csv and computes a new score that blends the base score with accumulated user ratings.

**update\_scores\_in\_materials(materials\_df) Function:**

Iterates over every material in the DataFrame and replaces each material's score with its newly calculated adaptive score. This function is called at the beginning of every recommendation cycle, ensuring the system always reflects the most up-to-date learning state.

**get\_feedback\_summary() Function:**

Generates a summary report grouping all collected feedback by material title, displaying the average rating and total count of ratings for each material. This serves as the "learning progress" view accessible from the main menu.

2.2.5 utils.py — Shared Utility Layer

utils.py provides a collection of **cross-cutting utility functions** that are shared across all other modules, preventing code duplication and ensuring consistency.

| Function                      | Purpose                                                                                            |
|-------------------------------|----------------------------------------------------------------------------------------------------|
| clear_screen()                | Clears the terminal for a clean, professional CLI output.                                          |
| print_banner()                | Renders the application's ASCII art title banner.                                                  |
| print_separator()             | Prints a 60-character horizontal separator line.                                                   |
| get_timestamp()               | Returns the current datetime as a formatted string (YYYY-MM-DD HH:MM:SS).                          |
| validate_rating(rating_input) | Validates that a user-entered rating is an integer between 1 and 5; returns None on invalid input. |

| Function                              | Purpose                                                                                                    |
|---------------------------------------|------------------------------------------------------------------------------------------------------------|
| load_csv(filepath)                    | Safely loads a CSV file into a Pandas DataFrame, handling FileNotFoundError and EmptyDataError gracefully. |
| display_materials_table(materials_df) | Renders recommended materials in a numbered, column-aligned tabular format in the CLI.                     |

### 3. AI/ML Algorithms & Formulas

This section provides a rigorous technical explanation of the two core AI/ML methodologies implemented in the project.

#### 3.1 Search Strategy: Constraint-Based Filtering

##### Conceptual Basis:

The recommendation process begins with a **constraint satisfaction** step. In classical AI, a constraint satisfaction problem (CSP) involves finding a state that satisfies a set of constraints. Here, the "state" is a set of study materials, and the constraints are the student's expressed preferences.

##### Implementation:

The filtering operation in recommender.py is semantically equivalent to the following SQL query:

```
sql
```

```
SELECT *
```

```
FROM materials
```

```
WHERE subject = :student_subject
```

```
AND difficulty = :student_difficulty
```

```
AND type = :student_type
```

In Python/Pandas, this is implemented as:

```
python
```

```
filtered = materials_df[
```

```
(materials_df["subject"].str.lower() == subject.lower()) &
```

```
(materials_df["difficulty"].str.lower() == difficulty.lower()) &
(materials_df["type"].str.lower() == mat_type.lower())
].copy()
```

### Properties:

- **Exact-match filtering** ensures that only materials precisely matching all three constraints are considered, maintaining the semantic integrity of the recommendation.
- The use of `.str.lower()` for case-insensitive comparison prevents trivially avoidable mismatches.
- The filtered subset serves as the candidate pool for the subsequent ranking step.

---

## 3.2 Heuristic Ranking Formula

### Conceptual Basis:

A **heuristic** in AI is an informed estimation function that guides decision-making when an optimal solution is computationally expensive or unknown. In this system, the heuristic quantifies the *predicted utility* of a material for a given student query. Materials with higher heuristic scores are assumed to be more beneficial and are ranked accordingly.

### Formula:

$$\text{Final Score} = \text{adaptive\_score} \times \text{difficulty\_weight} \times \text{type\_weight}$$

### Weight Tables:

| Difficulty | Weight | Rationale                                                               |
|------------|--------|-------------------------------------------------------------------------|
| Easy       | 1.0    | Baseline; foundational materials carry standard weight.                 |
| Medium     | 1.1    | A 10% boost; intermediate content balances challenge and comprehension. |
| Hard       | 1.2    | A 20% boost; rewards engagement with rigorous, high-effort material.    |

| Material Type | Weight | Rationale                                                                |
|---------------|--------|--------------------------------------------------------------------------|
| Video         | 1.0    | Baseline; passive consumption format.                                    |
| Notes         | 1.05   | A 5% boost; concise, referenceable reference material.                   |
| Practice      | 1.1    | A 10% boost; active learning and problem-solving yield deeper retention. |

### Worked Example:

Consider a material with an adaptive score of **4.2**, difficulty **Hard**, and type **Practice**:

Final Score =  $4.2 \times 1.2 \times 1.1 = \mathbf{5.544}$

Contrast with a material having an adaptive score of **4.5**, difficulty **Easy**, and type **Video**:

Final Score =  $4.5 \times 1.0 \times 1.0 = \mathbf{4.500}$

Despite having a higher adaptive score, the second material ranks *lower* because the heuristic weights favour challenging, active-learning content — reflecting a pedagogically informed design choice.

### Implementation:

python

```
def calculate_heuristic_score(adaptive_score, difficulty, mat_type):
```

```
    d_weight = DIFFICULTY_WEIGHTS.get(difficulty, 1.0)
```

```
    t_weight = TYPE_WEIGHTS.get(mat_type, 1.0)
```

```
    heuristic_score = adaptive_score * d_weight * t_weight
```

```
    return round(heuristic_score, 3)
```

---

## 3.3 Adaptive Learning — Reinforcement-Inspired Feedback

### Conceptual Basis:

**Reinforcement Learning (RL)** is a paradigm in which an agent learns to improve its decisions by receiving reward signals from the environment. This project implements a **simplified, explicit analogue** of RL: the user's numerical rating constitutes the "reward signal," and the system adjusts material scores accordingly. Materials that

consistently receive high ratings are surfaced more prominently in future sessions; poorly-rated materials are demoted.

This mechanism is philosophically similar to the **Bayesian update** process: the system begins with a prior belief (the `base_score`) and updates it iteratively as new evidence (user ratings) arrives.

#### Adaptive Score Formula:

$$\text{adaptive\_score} = \frac{\text{base\_score} + \sum_{i=1}^N r_i}{1 + N}$$

Where:

| Symbol                      | Definition                                                                                |
|-----------------------------|-------------------------------------------------------------------------------------------|
| <code>base_score</code>     | The initial score assigned to a material at dataset creation time.<br>Default: <b>3.5</b> |
| $r_i$                       | The $i$ -th user rating for this material (integer, 1–5).                                 |
| $N$                         | The total number of ratings collected for this material.                                  |
| <code>adaptive_score</code> | The recalculated score used in all future recommendation cycles.                          |

#### Analysis of Formula Behaviour:

- **No feedback ( $N = 0$ ):** `adaptive_score` = `base_score` / 1 = `base_score`. The formula is identity-preserving; without data, no change occurs.
- **Denominator ( $1 + N$ ):** The +1 prevents division by zero and acts as a **Laplace smoothing** factor, preventing the score from shifting too radically on a single rating — a well-established technique in probabilistic estimation.
- **Convergence property:** As  $N$  grows large, the influence of `base_score` diminishes and the score converges toward the true mean of user ratings, i.e.:  $\lim_{N \rightarrow \infty} \text{adaptive\_score} = \frac{\sum r_i}{N} = \bar{r}$

#### Worked Example:

A material has `base_score` = 3.5 and has received three ratings: **[5, 4, 5]**.

$$\text{adaptive\_score} = (3.5 + 5 + 4 + 5) / (1 + 3) = 17.5 / 4 = \mathbf{4.375}$$

On the next recommendation cycle, this material's score of **4.375** (instead of the original **3.5**) is fed into the heuristic ranking formula, causing it to rank higher than before — the system has **learned** that students find this material highly valuable.

#### Implementation:

python

```
def calculate_adaptive_score(material_id, base_score):  
    # ...load feedback_df from feedback.csv...  
    material_feedback = feedback_df[feedback_df["material_id"] == material_id]  
    sum_ratings = material_feedback["rating"].sum()  
    count_ratings = len(material_feedback)  
    adaptive_score = (base_score + sum_ratings) / (1 + count_ratings)  
    return round(adaptive_score, 2)
```

---

---

## 4. Implementation Details

### 4.1 Technology Stack

| Technology                    | Version | Role                                                           |
|-------------------------------|---------|----------------------------------------------------------------|
| <b>Python</b>                 | 3.x     | Primary programming language                                   |
| <b>Pandas</b>                 | Latest  | CSV data loading, DataFrame manipulation, groupby aggregations |
| <b>OS (stdlib)</b>            | —       | Cross-platform terminal clearing (cls / clear)                 |
| <b>Datetime (stdlib)</b>      | —       | Timestamping feedback records                                  |
| <b>Command Line Interface</b> | —       | User interaction medium                                        |

#### Rationale for Technology Choices:

- **Python** was selected for its readability, extensive standard library, and the availability of Pandas for structured data operations.
- **Pandas** provides an efficient, expressive API for dataset filtering and aggregation that mirrors the logic of relational database queries.
- **No advanced ML libraries** (e.g., scikit-learn, TensorFlow) were used. This was a deliberate design choice to ensure the AI/ML concepts are implemented transparently and are fully understandable without treating algorithms as black boxes.

- **CSV-based persistence** was chosen for simplicity and portability; the data files are human-readable and require no database server to operate.

---

#### 4.2 Data Structure: data/materials.csv

The primary dataset, materials.csv, stores all available study materials. Each row represents one material resource with the following schema:

| Column     | Data Type        | Description                                                                                                                                                                             |
|------------|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| id         | String / Integer | Unique identifier for the material. Used as the foreign key in feedback.csv.                                                                                                            |
| title      | String           | Descriptive title of the study material.                                                                                                                                                |
| subject    | String           | Academic subject (e.g., Python, Data Structures, Machine Learning).                                                                                                                     |
| difficulty | String           | Difficulty tier: Easy, Medium, or Hard.                                                                                                                                                 |
| type       | String           | Material type: Video, Notes, or Practice.                                                                                                                                               |
| base_score | Float            | Initial relevance score assigned at dataset creation. Default: 3.5.                                                                                                                     |
| score      | Float            | Runtime score column; populated dynamically during each session with the adaptive score. <b>This column is not stored persistently in the CSV</b> — it is computed in-memory each time. |

---

#### 4.3 Data Structure: data/feedback.csv

The feedback.csv file serves as the system's **long-term memory**, persisting all user ratings across sessions. Each row represents one rating event.

| Column      | Data Type        | Description                                    |
|-------------|------------------|------------------------------------------------|
| material_id | String / Integer | Foreign key linking to materials.csv.          |
| title       | String           | Material title (denormalised for readability). |
| subject     | String           | Subject of the rated material.                 |

| Column     | Data Type | Description                                                  |
|------------|-----------|--------------------------------------------------------------|
| difficulty | String    | Difficulty of the rated material.                            |
| type       | String    | Type of the rated material.                                  |
| rating     | Integer   | User-assigned rating (1 = Poor, 5 = Excellent).              |
| timestamp  | String    | ISO-format datetime string of when the rating was submitted. |

**Design Note:** The feedback file is append-only. All historical ratings are retained, which allows the adaptive formula to consider the *complete history* of a material's reception. This mirrors how recommendation systems accumulate evidence over time.

---

#### 4.4 Project Directory Structure

Study\_Recommender/

```

|
|— main.py      # Entry point and session loop controller
|— agent.py     # StudyAgent class (Perceive–Action–Learn cycle)
|— recommender.py # Constraint search + heuristic ranking engine
|— feedback.py  # Adaptive learning engine + feedback persistence
|— utils.py     # Shared utility functions
|
└─ data/
    └─ materials.csv # Study material dataset (static, curated)
    └─ feedback.csv  # User feedback log (dynamic, append-only)

```

---

## 5. Conclusion & Future Scope

### 5.1 Conclusion

This project successfully demonstrates that **meaningful AI/ML behaviour can be implemented from first principles**, without reliance on high-level machine learning

frameworks. The AI Adaptive Study Material Recommender System achieves the following:

- **Intelligent Agent Design:** The StudyAgent class faithfully models the classical agent loop — perceiving environmental stimuli, processing inputs through a reasoning layer, acting upon the environment, and updating its knowledge via feedback.
- **Constraint-Based Search:** Efficient, semantically precise filtering of the study material database, ensuring recommendations are always contextually appropriate.
- **Heuristic Ranking:** A transparent, interpretable scoring formula that encodes pedagogical intuitions (rewarding difficulty and active learning) into the recommendation order.
- **Adaptive Learning:** A statistically grounded score recalibration mechanism that causes the system to genuinely improve with use, demonstrating a core tenant of intelligent systems — the ability to learn from experience.
- **Robust CLI:** A clean, validated, and user-friendly command-line interface with persistent feedback storage across sessions.

The project bridges theoretical AI/ML concepts — agent models, heuristic functions, and reinforcement-inspired feedback — and concrete, working software, making it a strong pedagogical exercise in applied artificial intelligence.

---

## 5.2 Limitations

- The constraint-based search strategy requires an **exact match** across all three parameters simultaneously. Students who do not know the precise subject name string available in the dataset may receive no results.
  - The dataset is **static and manually curated**, limiting scalability.
  - The heuristic weights are **hand-crafted constants** rather than learned parameters, and may not generalise optimally across all student populations.
  - The system does not model **individual student profiles** — all users contribute to and benefit from the same global feedback pool.
- 

## 5.3 Future Scope

The following enhancements represent natural extensions of the current system:

1. **Collaborative Filtering:** Introduce user profiles and apply collaborative filtering to generate personalised recommendations based on shared preferences among students with similar profiles.
2. **Content-Based Filtering with NLP:** Analyse material descriptions using TF-IDF or sentence embeddings to enable semantic search beyond exact keyword matching.
3. **Dynamic Weight Learning:** Replace hand-coded heuristic weights with a **gradient-descent optimised** weight vector, learned iteratively from the accumulated feedback data.
4. **Web / GUI Interface:** Migrate from the CLI to a web-based frontend (Flask/Streamlit) for broader accessibility and an improved user experience.
5. **Extended Dataset:** Integrate with external APIs (e.g., YouTube Data API, open courseware repositories) to dynamically populate and refresh the materials database.
6. **Temporal Decay:** Introduce a time-decay factor in the adaptive formula, giving more weight to recent ratings than older ones — better reflecting the evolving quality and relevance of materials.
7. **Multi-Criteria Feedback:** Extend the rating system to collect multi-dimensional feedback (e.g., separate ratings for clarity, difficulty accuracy, and content quality), enabling a richer learning signal.
8. **A/B Testing Framework:** Implement experimental evaluation to empirically measure the impact of the adaptive learning mechanism on recommendation quality over time.

---

— End of Report —

**Author:** Sayan Mondal | B.Tech CSE (AIML), 1st Year | VIT Bhopal University | March 2026